Assignment Number 4 Documentation

Multi-Application Blazor Server Development Suite

Student Name: Awaab

Student Email: 247371@students.au.edu.pk

Instructor: Mr. Qaiser Ali

Department: Computer Science

Chairperson: Dr. Abdul Hameed

GitHub Repository: Awaab21/assignment4

ðŸ› ï¸ Architectural Overview

The workspace has been organized into **8 standalone Blazor Web App projects** compiling and serving independently. Each project implements a dedicated assignment application. A unified glassmorphic sidebar menu is embedded in all layouts to enable quick navigation across active local ports.

Local Hosting Port Matrix

- **Application01 (Data Binding):** Port 5001
- **Application02 (Form Validation):** Port 5002
- **Application03 (Local To-Do):** Port 5003
- **Application04 (Click Counter):** Port 5004
- **Application05 (Authentication State):** Port 5005

- **Application06 (DI Notifications):** Port 5006
- **Application07 (Theme Toggler App):** Port 5007
- **Application08 (Database To-Do Sync):** Port 5008 (MS SQL LocalDB)

ðŸ”— Application 01: Data Binding Demonstration

App 1 (Port 5001)

This application demonstrates the core differences between C# one-way and two-way data binding models in Blazor Server. The text input captures username via two-way binding, triggering immediate re-renders of the one-way bound greeting header.

Core Page Implementation Code (Application01/Components/Pages/Home.razor)

```
@page "/"
@rendermode InteractiveServer

App 1: Data Binding Demo

Explore the differences between one-way and two-way data binding in
Blazor.

Enter your name (Two-Way Bound):
Type a name here...
```

```
        ðŸ'‹

            Greeting (One-Way Bound):
            @@if (string.IsNullOrWhiteSpace(userName)) {
                Hello, Guest! Please enter your name above.
            } else {
                Hello, @@userName! Welcome to our premium Blazor das
hboard.
            }




@@code {
    private string userName { get; set; } = string.Empty;
}
```

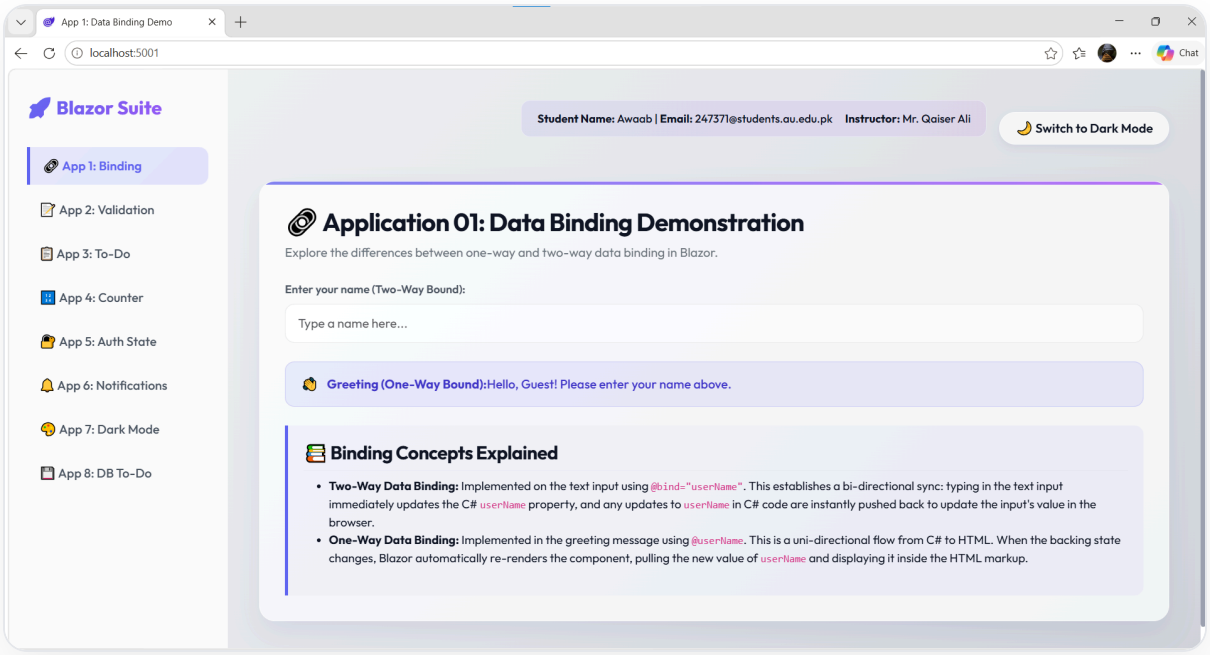


Figure 1: Application 01 User Interface showcasing data binding inputs.

ðŸ“ Application 02: Form Validation with Data Annotations

App 2 (Port 5002)

This component validates a registration form using C# Data Annotation properties inside an `EditForm` context. It checks presence using `[Required]` and verifies structure via `[EmailAddress]`.

Core Page Implementation Code (Application02/Components/Pages/Home.razor)

```
@page "/"
@rendermode InteractiveServer
@using System.ComponentModel.DataAnnotations

App 2: Form Validation Demo









A styled user registration form validating required inputs and email formatting.


    @@if (formSubmittedSuccessfully) {


            ðŸŽ‰
```

```
                Form Submission Success!

                The user account for @@lastSubmittedModel.FirstName
@@lastSubmittedModel.LastName (@@lastSubmittedModel.Email) has been
successfully created.




    }
```

First Name *

Last Name *

Email Address *

Submit Registration

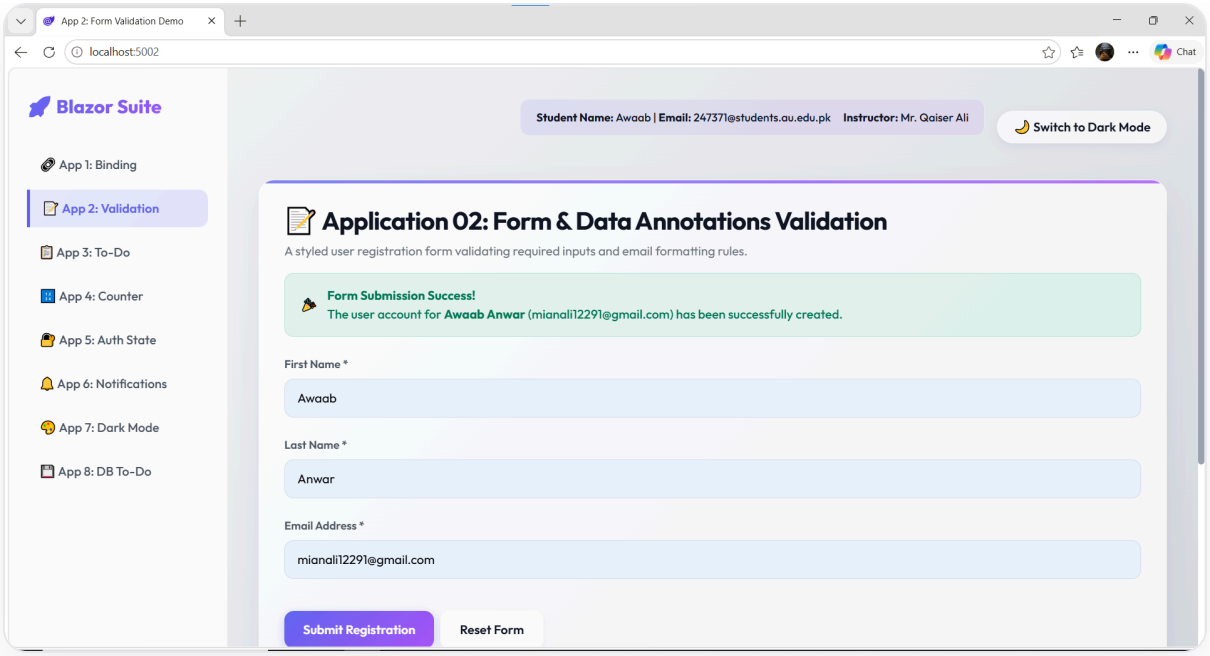


Figure 2: Form Validation warnings and error messaging display.

ðŸ“‹ Application 03: In-Memory To-Do List

App 3 (Port 5003)

Implements an in-memory task planner. A text box collects inputs with bidirectional synchronization, while a list processes changes dynamically in memory.

Core Page Implementation Code (Application03/Components/Pages/Home.razor)

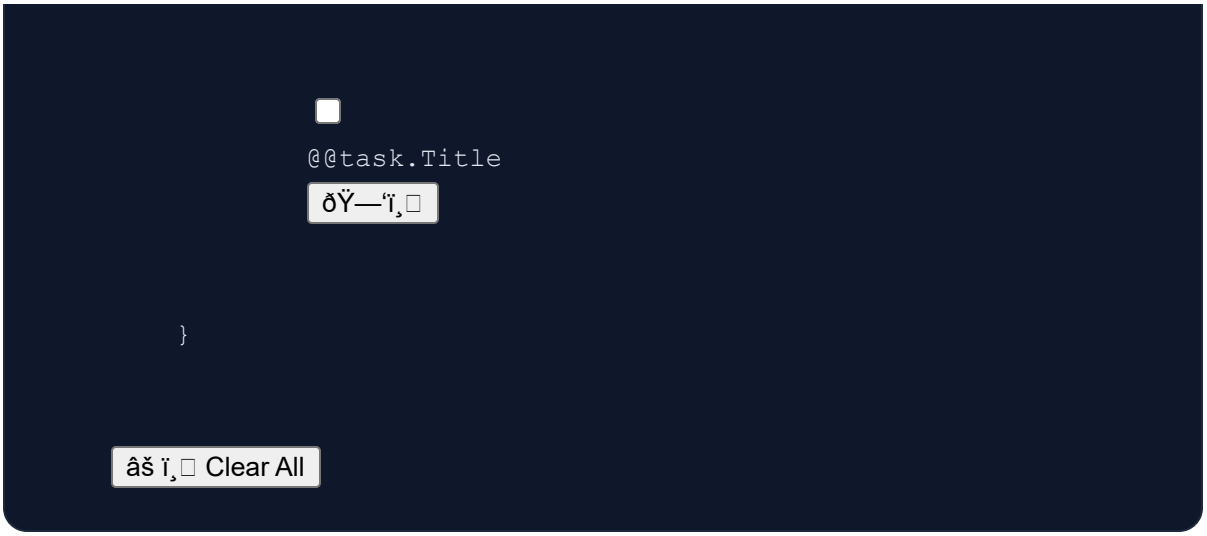
```
@page "/"
@rendermode InteractiveServer

App 3: Local To-Do List

Create and manage a quick local checklist.

Enter task...
Add Task

@@foreach (var task in tasks) {
```



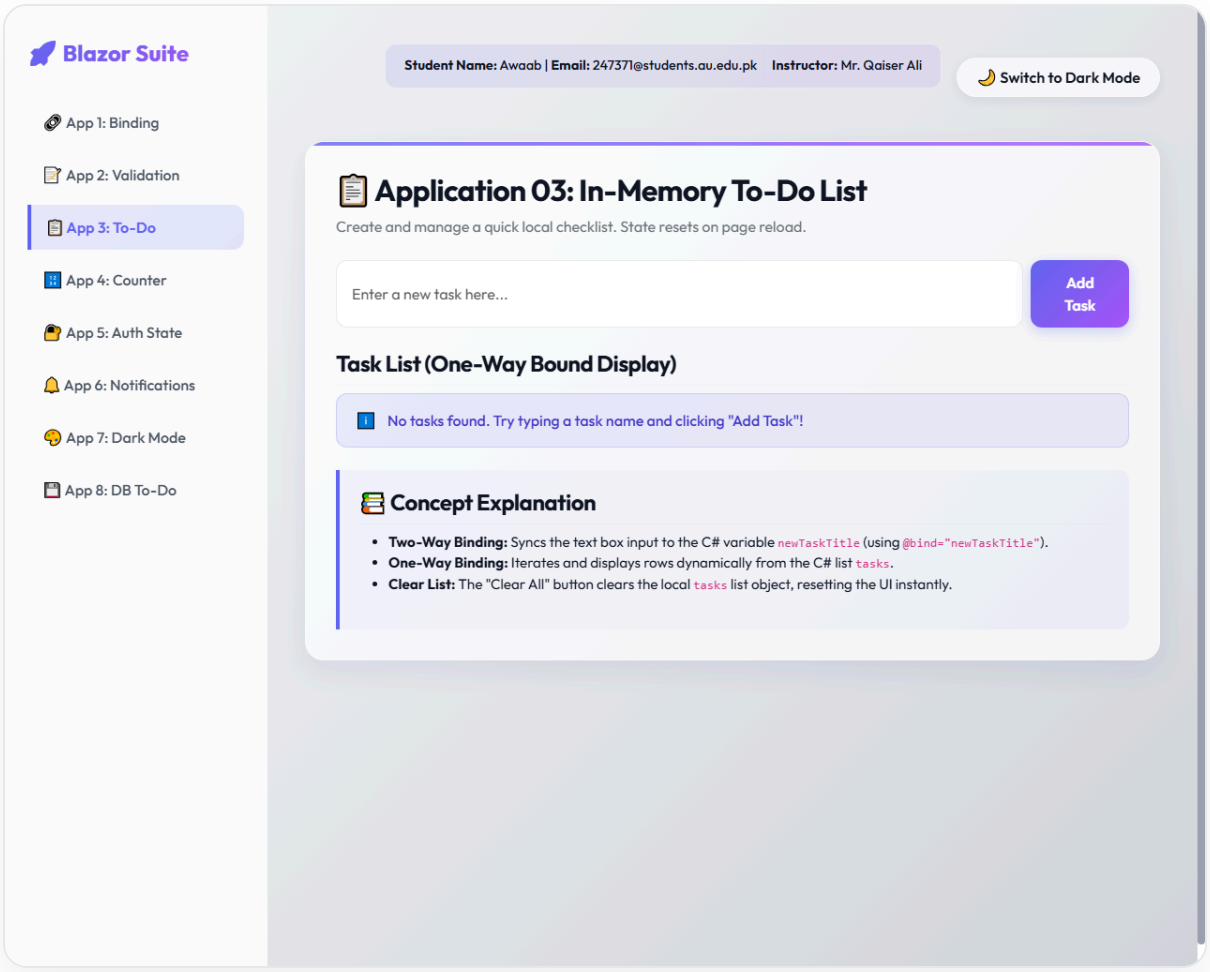


Figure 3: In-memory task list showing inputs and clear actions.

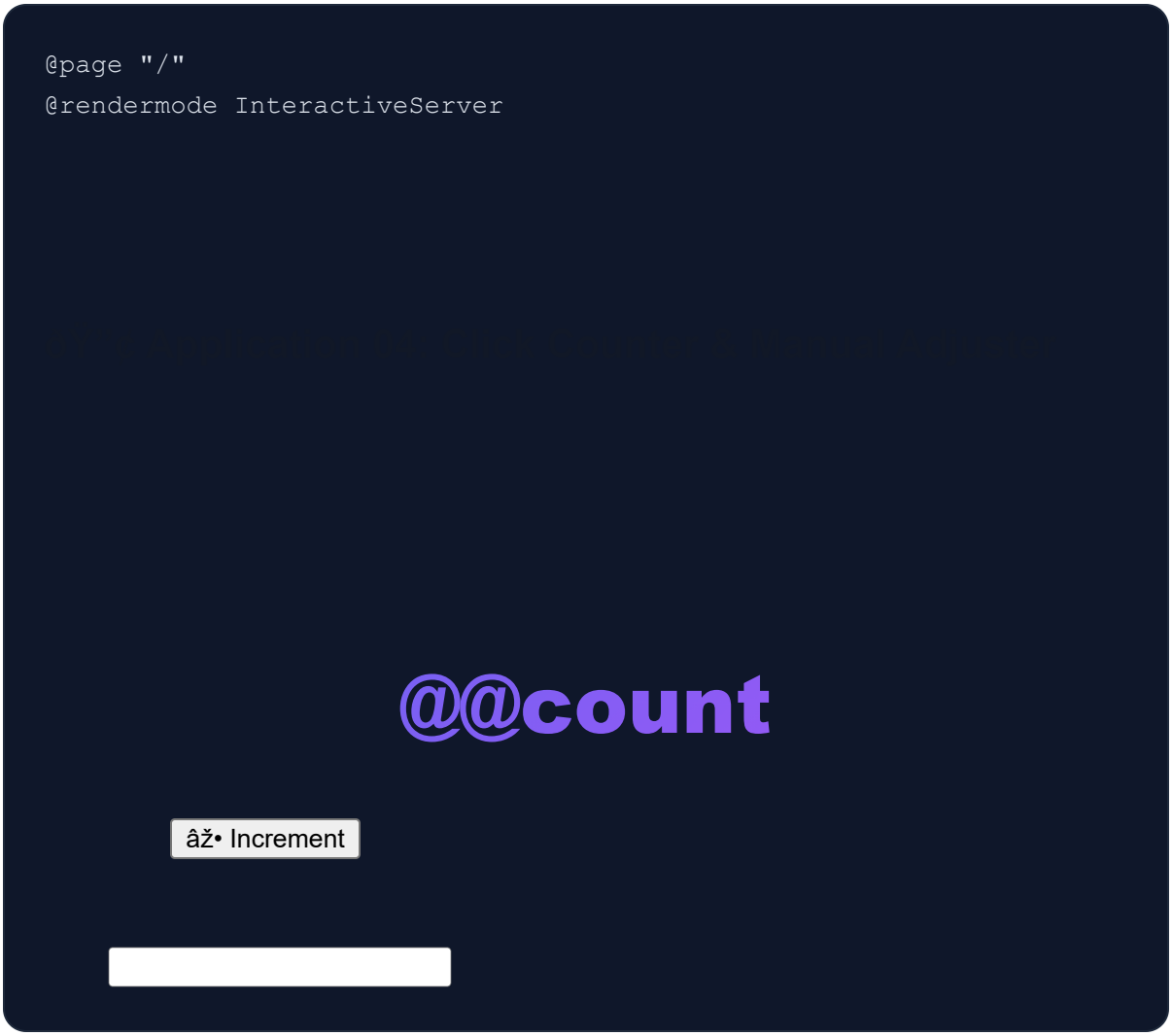
ðŸ”¢ Application 04: Click Counter & Manual Adjuster

App 4 (Port 5004)

Demonstrates basic event handler routines and combined data bindings. An `@@onclick` handler increments click numbers, and a number input allows direct manual settings using two-way bindings.

Core Page Implementation Code (Application04/Components/Pages/Home.razor)

```
@page "/"
@rendermode InteractiveServer
```



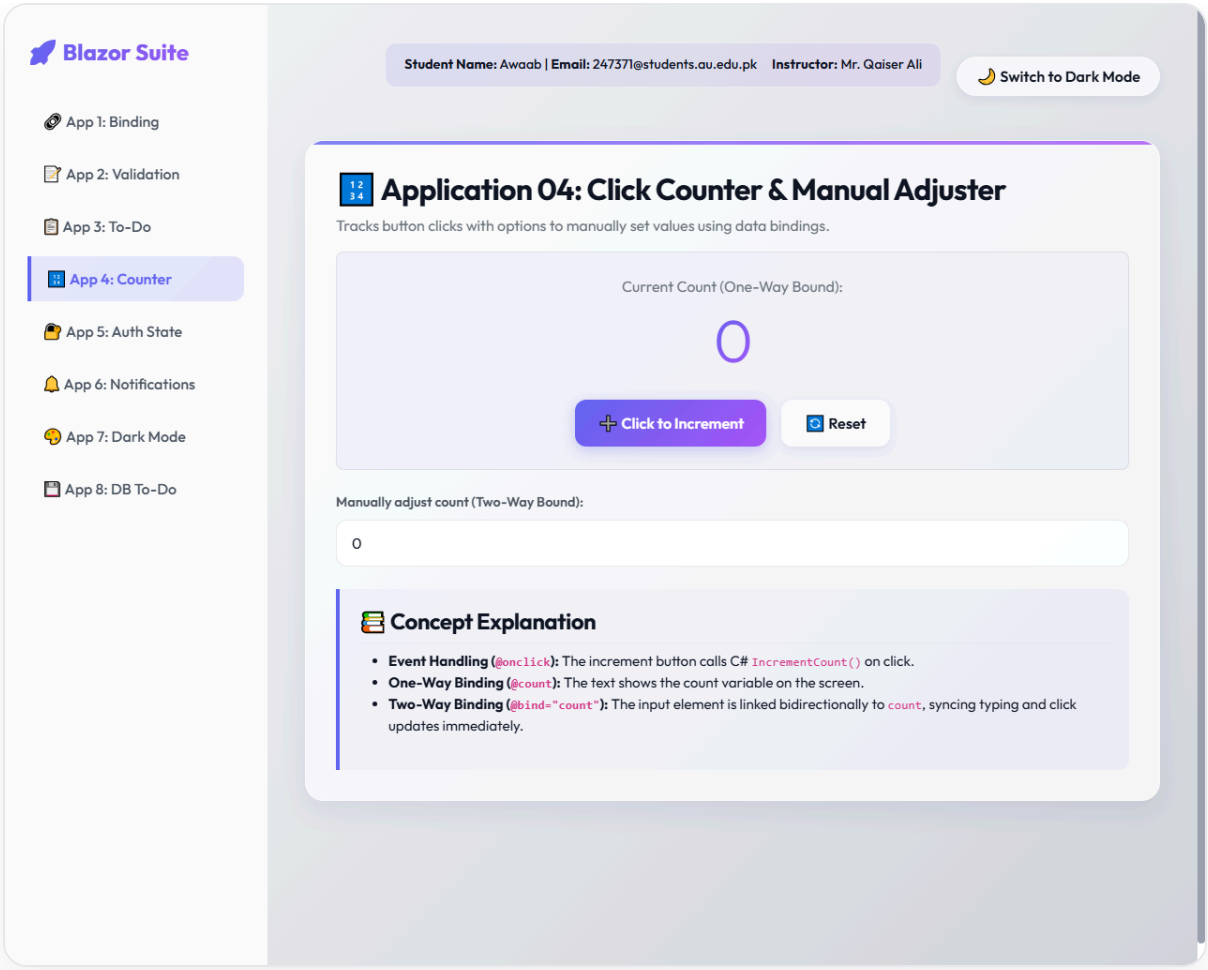


Figure 4: Dynamic counter displaying click calculations and input bindings.

ðŸ”□ Application 05: Singleton State Management

App 5 (Port 5005)

Demonstrates shared state tracking across independent UI widgets using a singleton service dependency. Toggling login inside the login widget triggers events that immediately update welcome banners in the profile panel.

C# Shared Authentication Service Code (Application05/Services/AuthenticationStateService.cs)

```
using System;

namespace Application05.Services
{
    public class AuthenticationStateService
    {
        public bool IsAuthenticated { get; private set; }
        public event Action? OnChange;

        public void LogIn() { IsAuthenticated = true; NotifyStateCha
nged(); }
        public void LogOut() { IsAuthenticated = false; NotifyStateC
hanged(); }
        private void NotifyStateChanged() => OnChange?.Invoke();
    }
}
```

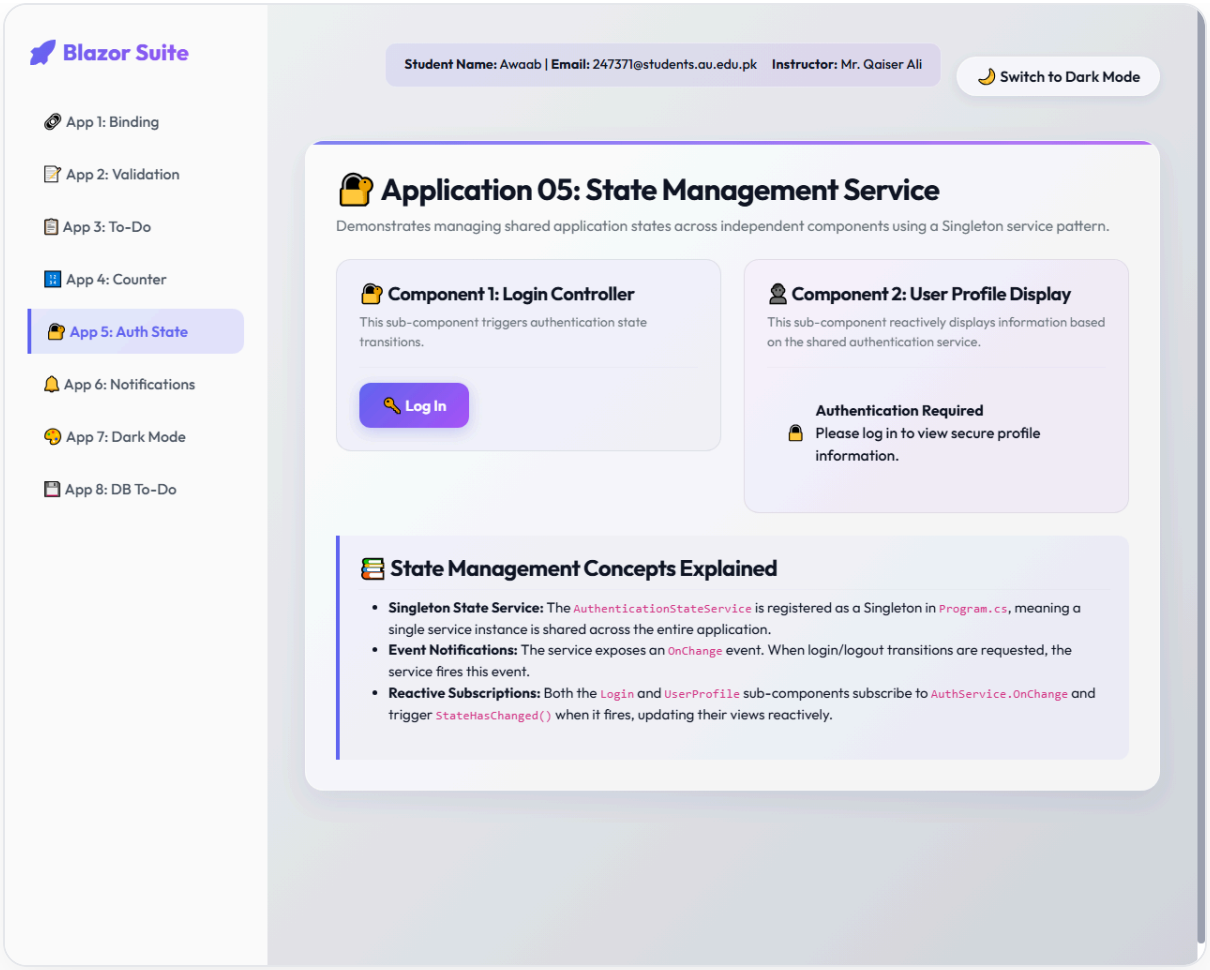


Figure 5: Singleton service propagating authentication status changes across components.

ðŸ”” Application 06: Dependency Injection & Configuration

App 6 (Port 5006)

Implements configuration setup and DI. A singleton configuration registers style and default count values. Settings modifications trigger notifications that dynamically flip presentation layout between Compact and Detailed cards.

DI Notification Service Class Code (Application06/Services/NotificationService.cs)

```
namespace Application06.Services
{
    public class NotificationService
    {
        private readonly NotificationConfig _config;
        public NotificationService(NotificationConfig config) { _con
fig = config; }

        public Task> GetNotificationsAsync(int? numberOfNotification
s = null) {
            int count = numberOfNotifications ?? _config.DefaultNumb
erOfNotifications;
            // Generates simulated entries...
        }
    }
}
```

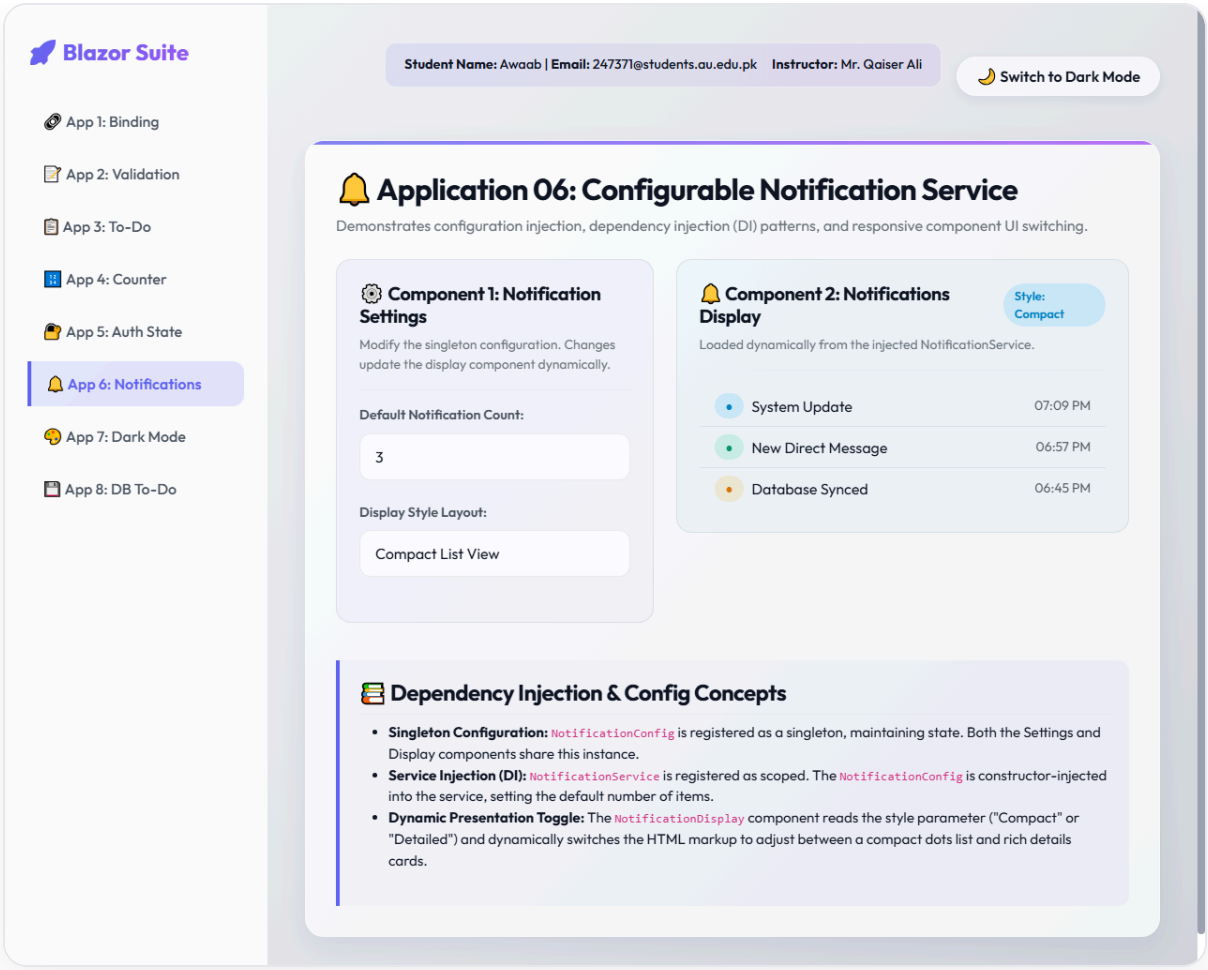


Figure 6: DI-configured settings panel shifting notification card details.

ðŸŽ¨ Application 07: Theme Switcher (Light/Dark Mode)

App 7 (Port 5007)

Toggles color schemes via JS Interop and persists selection in browser local storage. Variables adjust background gradients, cards, borders, text, and sidebars.

Theme helper implementation (Application07/wwwroot/js/theme.js)

```
window.themeHelper = {
    setTheme: function (theme) {
        localStorage.setItem('theme', theme);
        if (theme === 'dark') {
            document.body.classList.add('dark-theme');
            document.body.classList.remove('light-theme');
        } else {
            document.body.classList.add('light-theme');
            document.body.classList.remove('dark-theme');
        }
    },
    getTheme: function () { return localStorage.getItem('theme') ||
'light'; }
};
```

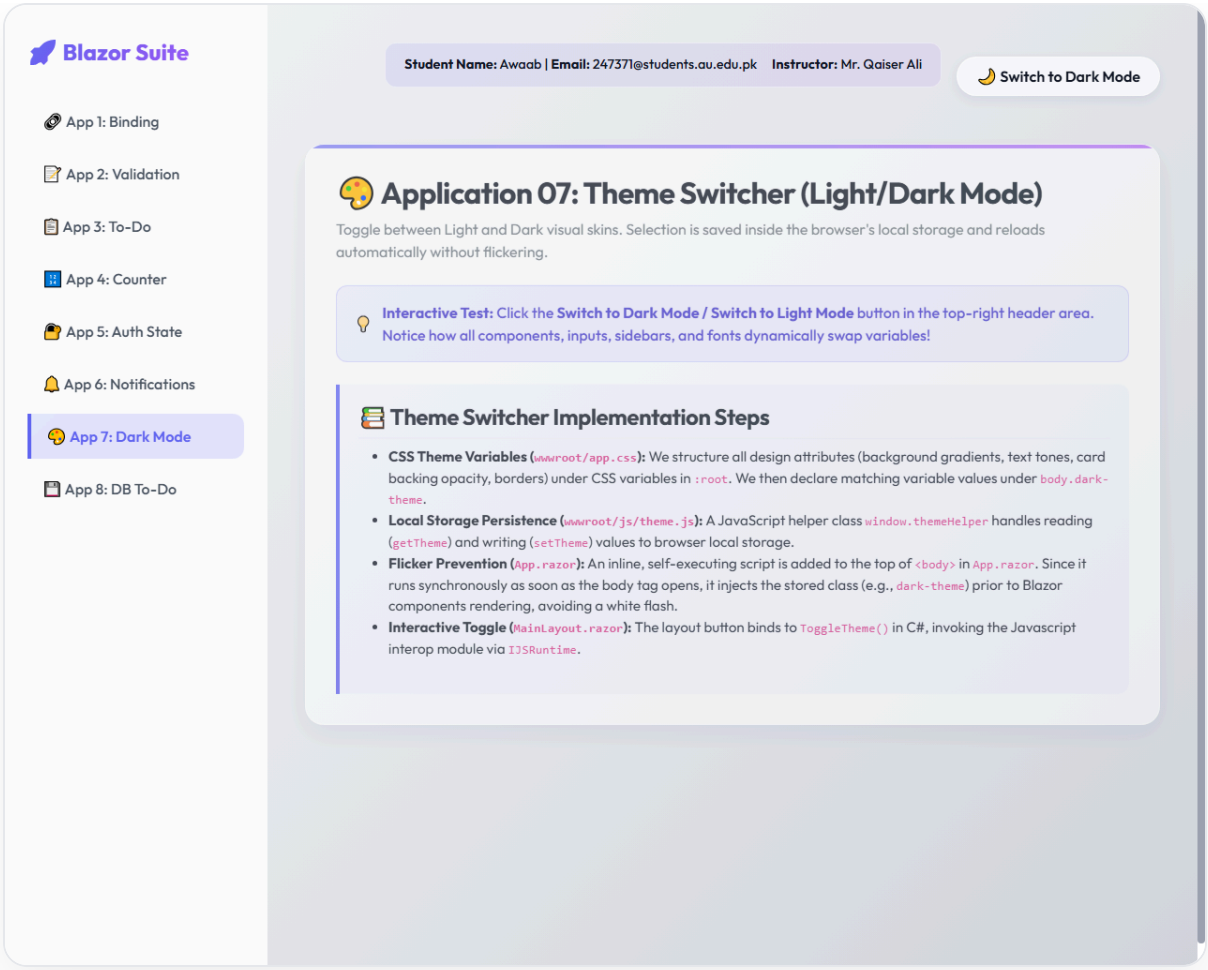


Figure 7: Responsive Light/Dark layout scheme showing custom dark parameters.

ðŸ“¦ Application 08: Database-Backed To-Do List

App 8 (Port 5008)

Connects the Client planner UI directly to MS SQL Server LocalDB. Add, toggle, delete, and clear actions perform database SQL commands and issue updates locally.

EF Core Database Context Code (Application08/Services/TodoDbContext.cs)

```
using Microsoft.EntityFrameworkCore;

namespace Application08.Services
{
    public class TodoDbContext : DbContext
    {
        public TodoDbContext(DbContextOptions options) : base(option
s) { }
        public DbSet TodoTasks { get; set; }
    }
}
```

Database Schema & Attributes Matrix

Column Name	Data Type	Constraints	Description
Id	INT	Primary Key, Identity(1,1)	Unique auto-incremented primary key.

Column Name	Data Type	Constraints	Description
Title	NVARCHAR(100)	Required, Not Null	The text title details of the task.
IsCompleted	BIT	Not Null	Completion boolean state flags.
CreatedAt	DATETIME2	Not Null	Time when record was logged.

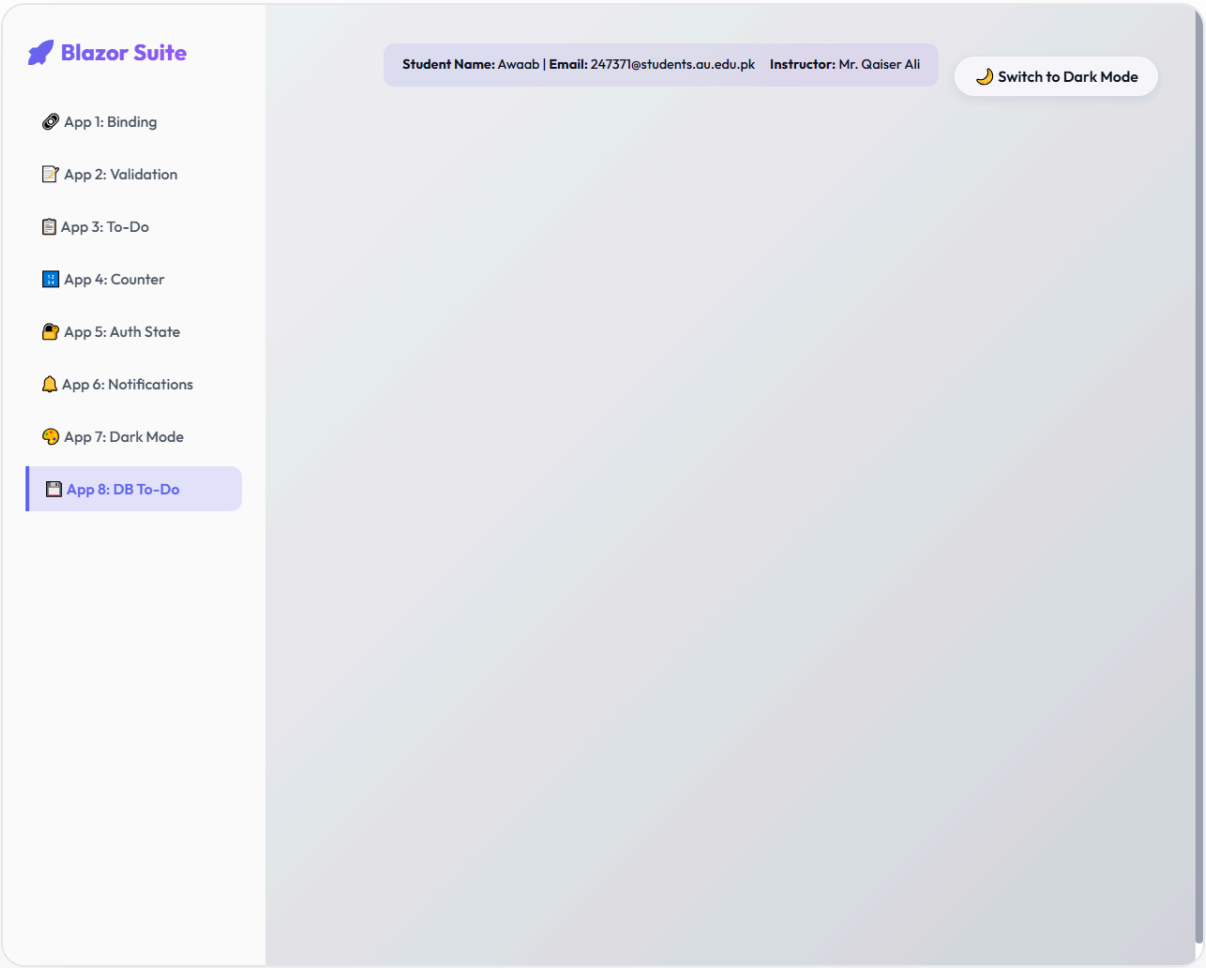


Figure 8: Database-synchronized list pulling elements directly from LocalDB SQL Server.